

UNIVERSITY OF MISSOURI - COLUMBIA

Final Project: Design and Simulation Vending Machine

Dillon Parrack and Christian Castellanos

ECE4250 - VHDL and Programmable Logic Devices

Date: 05/08/2025



Objective

The objective of this project is to simulate what a vending machine would do. This would allow a user to select an item, insert money, simulate the dispensing stage and return change. This would be achievable with seven segment displays, buttons, state machines, and multiplexers.

Background

Vending machines are famously known for their interactive sequencing of providing a user with their choice of product. Inspired by this multistep interaction and controlled sequence of events, our project sought to simulate the entire vending machine experience as a finite state machine (FSM) implemented in VHDL.

Finite State Machine

The process begins at the product selecting stage and will loop in this stage between 3 products until the 'select' button is pushed to exit out of the loop. When the product is selected, the price is recorded and used for a Boolean operation during the accumulator stage. In the initial design, the accumulator stage was comprised of a single state that would handle the accumulation of cents in increments of 25. However, limited by its simplicity, the state could not subtract and the arithmetic with a single state was rather challenging. To solve this, a unique state was declared for each possible value in the range of \$0.00 to \$3.00 in steps of \$0.25, resulting in a total of 13 states in the accumulator stage. This approach facilitated the arithmetic by adding 25 cents if moving to the next state or subtracting 25 cents if moving to a previous one. Unlike the product selection stage, there was no transition between the first and last states of the accumulator stage. To move on to the dispense stage, the accumulated money has to be equal or greater to the product price. If not, the state machine will remain in the accumulator stage until the Boolean is true. Next, the dispense state solely depended on a counter that would allow the machine to remain in that state for a determined period of time.

This simulated the time it takes for a real vending machine to dispense the item for the use to grab. Similarly, the change state was also counter dependent but also executed the necessary operations to display the difference between the accumulated money and the product price, which in turn gave the user its change. Figure 1 summarizes all the necessary states that were used for this simulation.

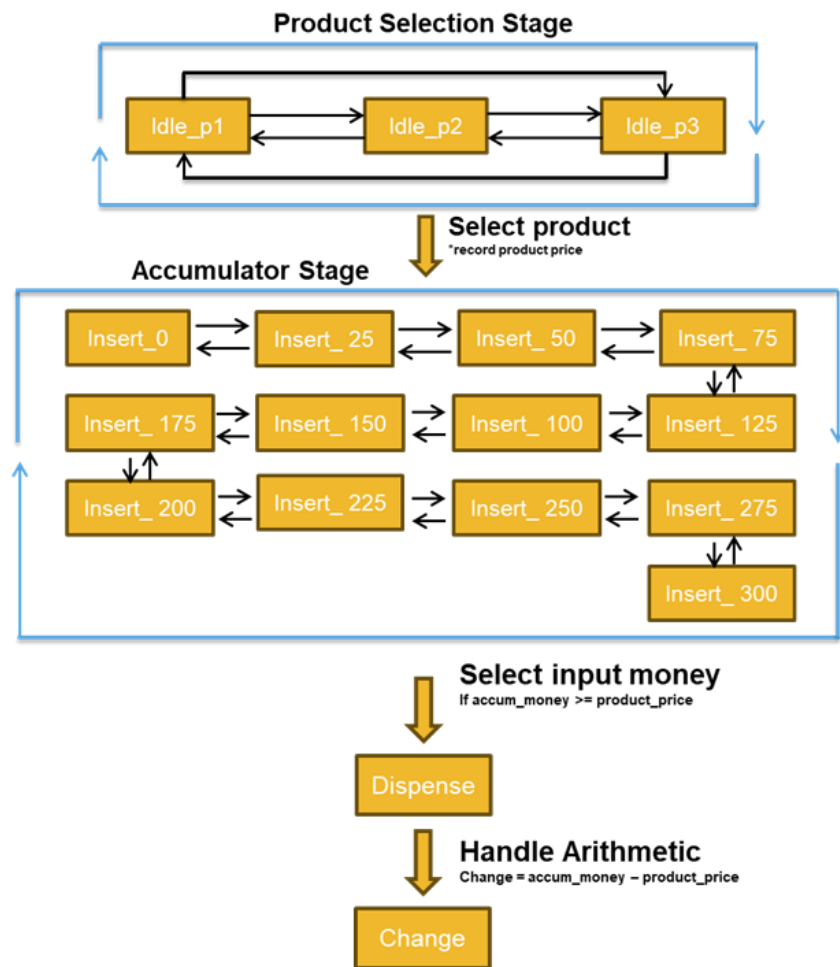


Figure 1. Vending machine top-level FSM sequence.

Methods

1.1 Lab dec_7seg.vhd Library

To display the necessary characters in the 7-segment LED display, it was necessary to understand the control signals that map the individual LED segments. Figure 2 explains the mapping and sequencing necessary to achieve different characters. From a previous lab, a file called dec-7seg which serves as a library for hexadecimal to 7-segment LED display conversion was used. The file was updated with the necessary letter to complete this project.

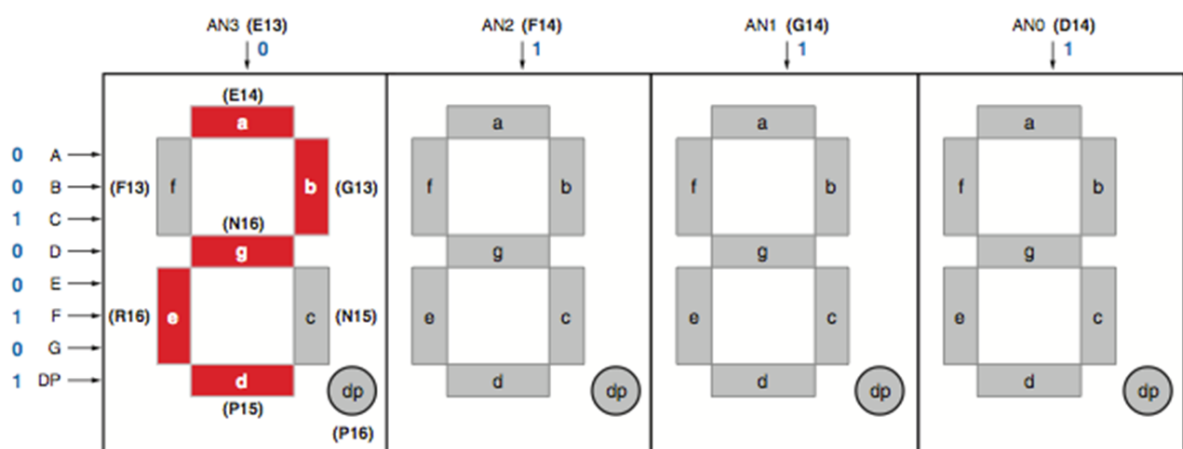


Figure 2. Control signals mapping to light individual LED segments.

1.2 Message Arrays

A very important approach for our project was the use of message arrays and accessing these during different states. These arrays consisted of stored messages in the form of 5-bits, where the first 4 bits routed to the dec_7seg to select the character and the least significant bit (LSB) determines if the decimal point is on ('0') or off ('1'). The message is then carried by *msg_t* which is an array that holds a total of 8 elements, each in the corresponding order of the LED (from left to right) that the character will be displayed in, e.g. ("10101", "00011", "11111", "00010", "01111", "01011", "11111", "11111") will display 'P1 1.75 '. Note that the displayed message included 2 empty LEDs indicated by '11111' which corresponds to all segments off.

To store all necessary display information, multiple message arrays were divided into stages that the state machine could reference to. First, *product_array* stores the messages to display the products and their prices plus the ‘*dispense*’ message to avoid declaring a single element array for the dispensing stage. Then, *accumulator_array* stores all the accumulated money values ranging from \$0.00 to \$3.00 in increments of 0.25. For simplicity and to avoid floating numbers, all numerical values used were integers and the ability of toggling the decimal point LED allowed for the correct representation of these integers into the expected cent values. Finally, *blank_message* turns off all displays.

1.3 Counters and Clocks

It is important to note that to be able to generate a clock, the clock frequency from the hardware being utilized is necessary to find the proper conversion formula. For the case of the Spartan 3 board, the clock frequency is 50MHz. To find the full clock period, the output clock should toggle every half period. A standard computation to obtain the desired clock period of 0.5 seconds is calculating the number of cycles for 0.25 seconds.

$$\text{Number of cycles (0.25 sec)} = \frac{0.25 \text{ sec}}{\frac{20 \times 10^{-9} \text{ sec}}{\text{cycle}}} = 12,500,000 \text{ cycles}$$

(1)

Formula 1 was used to find the number of cycles for the initial value of 0.5 seconds. A relationship between the number of cycles from Formula 1 and the rest of periods, 1, 2, and 4 seconds, was found and incorporated in the GenClock.vhd code. This relationship was found in Formula 2.

$$\text{Full Clock Period} = 12,500,000 \text{ cycles} \cdot 2^{(\text{time period})}$$

(2)

where *time period* is in the range of integers of 1 to 4, which yields the full clock period expected of 0.5s, 1s, 2s, 4s, respectively.

For this project it was only necessary to consider a 1 second clock period. A signal was declared to handle this clock inputs. The signal `clk_1sec` was used to flash the '*dispense*' message during the dispensing stage.

Due to the ongoing clock signal that drives the FPGA board, signals dependent on the GenClock create starting misalignments and inconsistent durations of states. The counters offered a better solution to this problem. To drive the dispense and change stages, it was necessary to declare local counters that would trigger when the next state started, guaranteeing synchronization and programmable time duration.

Code and Results

2.1 VHDL code for Vending Machine FSM

Our top-level work bench contains all the state machine logic. To start with this code, we first wanted to set the logic that would come into the main file. This included the buttons, motherboard clock, anodes for the seven-segment display, and the segments from the seven-segment display. We then created three different arrays to contain the messages that we would want to display, an array to turn off all the anodes when dispensing is flashing, and the different amounts that would be shown when a quarter is inserted. One key approach was the handling the arithmetic as integers to avoid floating numbers. For this we declared three constants for the price of the items available in the vending machine.

After declaring the signals, the main process in the project followed. This section includes the different button input logic. Depending on which button is pressed and the current state a unique message pertaining to that state would be displayed. If the user presses the middle button, it will record the price of the selected product and will then move to the insert money stage.

The up and down buttons logic was assigned for the money accumulator stage. If the up button is pressed, then the signal *money_inserted* would increment by 25 every time the button is pressed with a maximum value of 300. Conversely, if the down button is pressed then it will subtract the current value by 25 until reaching the initial value of 0. This summation will be displayed and updated after every button interaction. Once the required amount of money is inserted and the middle button is pressed, it will then flash '*dispense*' and handle the required arithmetic to display the correct amount of change.

Towards the end of the code, we can see that a certain message will display depending on which state the user is currently in. It also sets the refresh limit of the displays. The display message sent to *cs_nibble*, the section of the code that will process what the message will

display, and dp_bit, the section of the code that display a decimal point in a certain location on the seven-segment display.

We can then see that the code determines which anode is on, which segment is on, and which decimal point is on. The anodes work by seeing what number the bits are. The decimal point works by seeing if dp_bit is on ('0') or off ('1'). Finally, what determines which segments in th are on (0) or off (1) from the line of code that cs_nibble. When it comes to the line of code that cs_nibble contains the left side of the set of bits starts at segment_a and ends on the right side of the bits at segment_g.

2.2 VHDL code for dec_7seg

This file contains a library for the main file to refer to. Depending on the number that the bits are, that determines what will be displayed on the seven-segment display. This contains all the numbers from zero to nine. This also includes the letters P, n, C, D, E, H, and includes a unique sequence to turn off all the segments.

2.3 VHDL code for GenClock

This code provided insight on the synchronization of the states the user travelled with the FPGA's board internal clock. Using GenClock, it was possible to instantiate and obtain a signal that ran on a 1 second clock period. This was used for flashing the 'dispense' message to simulate a loading behavior in hardware, or in this case, simulate the dispensing stage of a real vending machine.

Discussion

3.1 7-Segment Display Challenges

One of the first challenges was learning how to properly operate the 7-segment display LEDs. Heavily referencing how lab 3 was set up, an initial control of the anodes was developed. Building on the original code, it was necessary to add more multiplexing counters to communicate with the rest of the available displays. After a number of unsuccessful attempts, most of the files for anode control and LED display were scratched and a new routing and multiplexing algorithm was employed. Keeping some key features from the lab 3 code, we eliminated the port mapping and included these algorithms directly in the FSM top-level work bench. After learning how to control the segments and the anodes we realized that we would need to turn each anode on and off rapidly to display a message. To overcome this, we created a slower clock with a refresh rate we could manipulate, and iterated different values to fine-tune the displayed messages. After many iterations and countless hours, the 7-segment displays worked.

3.2 Button Bouncing

During the initial button testing, the display would jump between messages. The signal was registering multiple “bounces” that are caused by the nature of spring found in mechanical buttons. Figure 3 depicts the sinusoidal behavior that comes from springs bouncing after pressing a button.

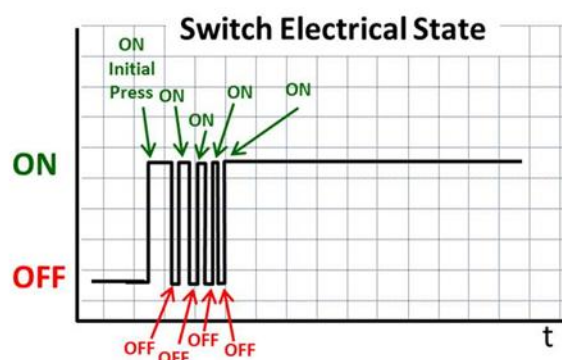


Figure 3. Sinusoidal behavior of spring-loaded buttons.

To overcome this issue, it was necessary to create a debouncing algorithm for the signal to be interpreted as a true single button input. For this, all repeating 'on' events within a timespan were considered and interpreted as a single signal.

Our approach was the implementation of a debounce using a counter. Each button required a separate signal that acted as a timer that would register the valid input. Using the FPGA board's 50 MHz clock and having the signal count once every clock tick ($1/50,000,000$ of a second), a counter threshold declaration followed. The counter threshold would encapsulate all participating 'on' events and register the valid input within that timespan. The initial threshold value was set to 30,000 which if dividing $30,000/50,000,000$, yields approximately 0.6 milliseconds. Stability of button input was found at a threshold value of 80,000, or about 1.6 milliseconds.

Conclusion

In conclusion, we were able to simulate a vending machine and add a level of complexity to the techniques employed during lab. Though challenging, the learning curve for the 7-segment display and debouncing algorithm made this project worthwhile. Besides the obstacles encountered, the hardware implementation of the Vending Machine FSM yielded the expected sequencing and behavior of the different mechanical components. This was proven by the correct button configuration, stage response, correct arithmetic at the change stage, and finally having the state machine successfully restart the process.

Appendix

VHDL code for GenClock

entity GenClock is

generic (time_period : integer range 1 to 4);

port (clk : in std_logic;

Clock : out std_logic);

end GenClock;

architecture Behavioral of GenClock is

constant max_count: integer := 12500000 * (2**(time_period));

signal count : integer range 0 to max_count := 0;

signal clk_out: std_logic := '0';

begin

process(clk)

begin

if (clk'event and clk = '1') then

if count = max_count then

count <= 0;

clk_out <= not clk_out;

else

count <= count + 1;

end if;

end if;

end process;

Clock <= clk_out;

end Behavioral;

VHDL code for Vending Machine FSM

entity VendingMachineFSM is

port (

clk : in std_logic;

btn_next : in std_logic;

btn_prev : in std_logic;

btn_select : in std_logic;

btn_coin_a : in std_logic;

btn_coin_s : in std_logic;

AN0,AN1,AN2,AN3,AN4,AN5,AN6,AN7 : out std_logic;

segment_a,segment_b,segment_c,

segment_d,segment_e,segment_f,

segment_g,segment_dp : out std_logic

```
);
end entity;
```

architecture top_level of VendingMachineFSM is

```
type msg_t is array(0 to 7) of std_logic_vector(4 downto 0);
type product_array_t is array(0 to 3) of msg_t;
type accumulator_array_t is array(0 to 12) of msg_t;
```

```
constant product_messages : product_array_t := (
  ("10101","00011","11111","00010","01111","01011","11111","11111"), --'P1 1.75'
  ("10101","00101","11111","00000","01101","01011","11111","11111"), --'P2 0.65'
  ("10101","00111","11111","00100","01001","00111","11111","11111"), --'P3 2.43'
  ("11011","00011","01011","10101","11101","10111","01011","11101") -- 'dispense'
);
```

```
constant PRICE_P1 : integer := 175;
constant PRICE_P2 : integer := 65;
constant PRICE_P3 : integer := 243;
```

```
constant accumulator_messages : accumulator_array_t := (
  ("11111","11111","11111","11111","11111","00000","00001","00001"), -- 0.00
  ("11111","11111","11111","11111","11111","00000","00101","01011"), -- 0.25
  ("11111","11111","11111","11111","11111","00000","01011","00001"), -- 0.50
  ("11111","11111","11111","11111","11111","00000","01111","01011"), -- 0.75
  ("11111","11111","11111","11111","11111","00010","00001","00001"), -- 1.00
  ("11111","11111","11111","11111","11111","00010","00101","01011"), -- 1.25
  ("11111","11111","11111","11111","11111","00010","01011","00001"), -- 1.50
  ("11111","11111","11111","11111","11111","00010","01111","01011"), -- 1.75
  ("11111","11111","11111","11111","11111","00100","00001","00001"), -- 2.00
  ("11111","11111","11111","11111","11111","00100","00101","01011"), -- 2.25
  ("11111","11111","11111","11111","11111","00100","01011","00001"), -- 2.50
  ("11111","11111","11111","11111","11111","00100","01111","01011"), -- 2.75
  ("11111","11111","11111","11111","11111","00110","00001","00001") -- 3.00
);
```

```
constant blank_message : msg_t := ("11111","11111","11111","11111","11111","11111","11111","11111");
```

```
type vending_state is (
  idle_p1, idle_p2, idle_p3,
  insert_0, insert_25, insert_50, insert_75,
  insert_100, insert_125, insert_150, insert_175,
  insert_200, insert_225, insert_250, insert_275, insert_300,
  dispensing, change
);
```

```
signal current_state : vending_state := idle_p1;
signal refresh_cnt : unsigned(11 downto 0) := (others => '0');
```

```

constant REFRESH_LIMIT : unsigned(11 downto 0) := to_unsigned(4095,12);

signal scan_idx : unsigned(2 downto 0) := (others => '0');
signal cs_nibble : std_logic_vector(3 downto 0);
signal dp_bit : std_logic;

signal selected_price : integer range 0 to 300 := 0;
signal money_inserted : integer range 0 to 300 := 0;

signal btn_next_cnt, btn_prev_cnt, btn_select_cnt : unsigned(16 downto 0) := (others => '0');
signal btn_coin_a_cnt, btn_coin_s_cnt : unsigned(16 downto 0) := (others => '0');
constant BTN_THRESHOLD : unsigned(16 downto 0) := to_unsigned(80000, 17);

signal clk_1sec : std_logic;
signal display_message : msg_t;

constant SEC : integer := 50000000; -- assuming 50 MHz clock
signal dispensing_counter : unsigned(28 downto 0) := (others => '0');
constant DISPENSING_TARGET : unsigned(28 downto 0) := to_unsigned(5 * SEC, 29);

signal change_counter : unsigned(28 downto 0) := (others => '0');
constant CHANGE_DURATION : unsigned(28 downto 0) := to_unsigned(10 * SEC, 29);
signal change_val : integer range 0 to 300 := 0;
signal change_digits : msg_t;
signal hundreds, tens, ones : integer range 0 to 9 := 0;

function digit_with_dp(val : integer; dp : std_logic) return std_logic_vector is
    variable result : std_logic_vector(4 downto 0);
begin
    result(4 downto 1) := std_logic_vector(to_unsigned(val, 4));
    result(0) := dp;
    return result;
end function;

begin

clk1_inst : entity work.GenClock generic map (time_period => 2) port map (clk, clk_1sec);

process(clk)
begin
    if rising_edge(clk) then

        -- Button debouncing and actions
        if btn_next = '1' then
            if btn_next_cnt < BTN_THRESHOLD then
                btn_next_cnt <= btn_next_cnt + 1;
            elsif btn_next_cnt = BTN_THRESHOLD then
                if current_state = idle_p1 then current_state <= idle_p2;
                elsif current_state = idle_p2 then current_state <= idle_p3;
            end if;
        end if;
    end if;
end process;

```

```

    elsif current_state = idle_p3 then current_state <= idle_p1;
    end if;
    btn_next_cnt <= btn_next_cnt + 1;
end if;
else
    btn_next_cnt <= (others => '0');
end if;

if btn_prev = '1' then
    if btn_prev_cnt < BTN_THRESHOLD then
        btn_prev_cnt <= btn_prev_cnt + 1;
    elsif btn_prev_cnt = BTN_THRESHOLD then
        if current_state = idle_p1 then current_state <= idle_p3;
        elsif current_state = idle_p2 then current_state <= idle_p1;
        elsif current_state = idle_p3 then current_state <= idle_p2;
        end if;
        btn_prev_cnt <= btn_prev_cnt + 1;
    end if;
else
    btn_prev_cnt <= (others => '0');
end if;

if btn_select = '1' then
    if btn_select_cnt < BTN_THRESHOLD then
        btn_select_cnt <= btn_select_cnt + 1;
    elsif btn_select_cnt = BTN_THRESHOLD then
        if current_state = idle_p1 then selected_price <= PRICE_P1; current_state <= insert_0;
money_inserted <= 0;
        elsif current_state = idle_p2 then selected_price <= PRICE_P2; current_state <= insert_0;
money_inserted <= 0;
        elsif current_state = idle_p3 then selected_price <= PRICE_P3; current_state <= insert_0;
money_inserted <= 0;
        elsif current_state >= insert_0 and current_state <= insert_300 then
            if money_inserted >= selected_price then
                current_state <= dispensing;
                dispensing_counter <= (others => '0');
            end if;
        end if;
        btn_select_cnt <= btn_select_cnt + 1;
    end if;
else
    btn_select_cnt <= (others => '0');
end if;

if btn_coin_a = '1' then
    if btn_coin_a_cnt < BTN_THRESHOLD then
        btn_coin_a_cnt <= btn_coin_a_cnt + 1;
    elsif btn_coin_a_cnt = BTN_THRESHOLD then
        if current_state >= insert_0 and current_state <= insert_300 then

```

```

if money_inserted < 300 then
    money_inserted <= money_inserted + 25;
    if current_state /= insert_300 then
        current_state <= vending_state'succ(current_state);
    end if;
end if;
end if;
btn_coin_a_cnt <= btn_coin_a_cnt + 1;
end if;
else
    btn_coin_a_cnt <= (others => '0');
end if;

if btn_coin_s = '1' then
    if btn_coin_s_cnt < BTN_THRESHOLD then
        btn_coin_s_cnt <= btn_coin_s_cnt + 1;
    elsif btn_coin_s_cnt = BTN_THRESHOLD then
        if current_state >= insert_0 and current_state <= insert_300 then
            if money_inserted > 0 then
                money_inserted <= money_inserted - 25;
                if current_state /= insert_0 then
                    current_state <= vending_state'pred(current_state);
                end if;
            end if;
        end if;
        btn_coin_s_cnt <= btn_coin_s_cnt + 1;
    end if;
else
    btn_coin_s_cnt <= (others => '0');
end if;

if current_state = dispensing then
    if clk_1sec = '1' then
        display_message <= product_messages(3);
    else
        display_message <= blank_message;
    end if;

    if dispensing_counter < DISPENSING_TARGET then
        dispensing_counter <= dispensing_counter + 1;
    else
        dispensing_counter <= (others => '0');
        change_val <= money_inserted - selected_price;
        change_counter <= (others => '0');
        hundreds <= (money_inserted - selected_price) / 100;
        tens    <= ((money_inserted - selected_price) mod 100) / 10;
        ones    <= (money_inserted - selected_price) mod 10;
        current_state <= change;
    end if;
end if;

```

```

elsif current_state = change then
  if change_counter < CHANGE_DURATION then
    change_counter <= change_counter + 1;
  else
    change_counter <= (others => '0');
    current_state <= idle_p1;
  end if;

  change_digits(0) <= "11001"; -- 'C'
  change_digits(1) <= "11001"; -- 'C'
  change_digits(2) <= "10111"; -- 'n'
  change_digits(3) <= "01101"; -- 'G'
  change_digits(4) <= "11111"; -- blank
  change_digits(5) <= digit_with_dp(hundreds, '0'); -- decimal point ON
  change_digits(6) <= digit_with_dp(tens, '1'); -- decimal point OFF
  change_digits(7) <= digit_with_dp(ones, '1'); -- decimal point OFF

  display_message <= change_digits;

else
  case current_state is
    when idle_p1 => display_message <= product_messages(0);
    when idle_p2 => display_message <= product_messages(1);
    when idle_p3 => display_message <= product_messages(2);
    when insert_0 => display_message <= accumulator_messages(0);
    when insert_25 => display_message <= accumulator_messages(1);
    when insert_50 => display_message <= accumulator_messages(2);
    when insert_75 => display_message <= accumulator_messages(3);
    when insert_100 => display_message <= accumulator_messages(4);
    when insert_125 => display_message <= accumulator_messages(5);
    when insert_150 => display_message <= accumulator_messages(6);
    when insert_175 => display_message <= accumulator_messages(7);
    when insert_200 => display_message <= accumulator_messages(8);
    when insert_225 => display_message <= accumulator_messages(9);
    when insert_250 => display_message <= accumulator_messages(10);
    when insert_275 => display_message <= accumulator_messages(11);
    when insert_300 => display_message <= accumulator_messages(12);
    when others => null;
  end case;
end if;

if refresh_cnt = REFRESH_LIMIT then
  refresh_cnt <= (others => '0');
  scan_idx <= scan_idx + 1;
else
  refresh_cnt <= refresh_cnt + 1;
end if;

```



```

cs_nibble <= display_message(to_integer(scan_idx))(4 downto 1);
dp_bit  <= display_message(to_integer(scan_idx))(0);

end if;
end process;

AN7 <= '0' when scan_idx = "000" else '1';
AN6 <= '0' when scan_idx = "001" else '1';
AN5 <= '0' when scan_idx = "010" else '1';
AN4 <= '0' when scan_idx = "011" else '1';
AN3 <= '0' when scan_idx = "100" else '1';
AN2 <= '0' when scan_idx = "101" else '1';
AN1 <= '0' when scan_idx = "110" else '1';
AN0 <= '0' when scan_idx = "111" else '1';

segment_dp <= dp_bit;

dec7 : entity work.dec_7seg
port map (
    current_state => cs_nibble,
    segment_a  => segment_a,
    segment_b  => segment_b,
    segment_c  => segment_c,
    segment_d  => segment_d,
    segment_e  => segment_e,
    segment_f  => segment_f,
    segment_g  => segment_g
);

end architecture top_level;

```

VHDL code for dec_7seg

```

entity dec_7seg is
port (
    current_state : in  std_logic_vector(3 downto 0);
    segment_a,segment_b,segment_c,
    segment_d,segment_e,segment_f,
    segment_g    : out std_logic
);
end entity;
architecture library of dec_7seg is
    signal segdata : std_logic_vector(6 downto 0);
begin
    process(current_state)
    begin
        case current_state is
            when "0000" => segdata <= "1111110"; -- 0
            when "0001" => segdata <= "0110000"; -- 1

```

```

when "0010" => segdata <= "1101101"; -- 2
when "0011" => segdata <= "1111001"; -- 3
when "0100" => segdata <= "0110011"; -- 4
when "0101" => segdata <= "1011011"; -- 5
when "0110" => segdata <= "1011111"; -- 6
when "0111" => segdata <= "1110000"; -- 7
when "1000" => segdata <= "1111111"; -- 8
when "1001" => segdata <= "1111011"; -- 9
when "1010" => segdata <= "1100111"; -- P
when "1011" => segdata <= "0010101"; -- n
when "1100" => segdata <= "1001110"; -- C
when "1101" => segdata <= "0111101"; -- D
when "1110" => segdata <= "1001111"; -- E
when "1111" => segdata <= "0000000"; -- blank
when others => segdata <= "0111011"; -- H
end case;
end process;
-- invert for common-anode
segment_a <= not segdata(6);
segment_b <= not segdata(5);
segment_c <= not segdata(4);
segment_d <= not segdata(3);
segment_e <= not segdata(2);
segment_f <= not segdata(1);
segment_g <= not segdata(0);
end architecture library;

```

Constraint File

Clock signal

```

set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports { clk }]; IO_L12P_T1_MRCC_35
Sch=clk #create_clock -name sys_clk_pin -period 20.000 -waveform {0 10} [get_ports {CLK100MHZ}];

```

##Buttons

```

set_property -dict { PACKAGE_PIN C12 IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN }];
IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resen; set_property -dict { PACKAGE_PIN N17 IOSTANDARD
LVCMOS33 } [get_ports { btn_select }]; IO_L9P_T1_DQS_14 Sch=btn_select; set_property -dict {
PACKAGE_PIN M18 IOSTANDARD LVCMOS33 } [get_ports { btn_coin_a }]; IO_L4N_T0_D05_14
Sch=btn_btn_coin_a; set_property -dict { PACKAGE_PIN P17 IOSTANDARD LVCMOS33 } [get_ports {
btn_prev}]; IO_L12P_T1_MRCC_14 Sch=btn_prev; set_property -dict { PACKAGE_PIN M17 IOSTANDARD
LVCMOS33 } [get_ports { btn_next }]; IO_L10N_T1_D15_14 Sch=btn_next; set_property -dict {
PACKAGE_PIN P18 IOSTANDARD LVCMOS33 } [get_ports { btn_coin_s }]; IO_L9N_T1_DQS_D13_14
Sch=btn_coin_s;

```

##7 segment display

```

set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports { segment_a }];
IO_L24N_T3_A00_D16_14 Sch=segment_a set_property -dict { PACKAGE_PIN R10 IOSTANDARD

```

LVC MOS33 } [get_ports { segment_b }]; IO_25_14 Sch=segment_b set_property -dict { PACKAGE_PIN K16
IOSTANDARD LVC MOS33 } [get_ports { segment_c }]; IO_25_15 Sch=segment_c set_property -dict {
PACKAGE_PIN K13 IOSTANDARD LVC MOS33 } [get_ports { segment_d }]; IO_L17P_T2_A26_15
Sch=segment_d set_property -dict { PACKAGE_PIN P15 IOSTANDARD LVC MOS33 } [get_ports { segment_e
}]; IO_L13P_T2_MRCC_14 Sch=segment_e set_property -dict { PACKAGE_PIN T11 IOSTANDARD
LVC MOS33 } [get_ports { segment_f }]; IO_L19P_T3_A10_D26_14 Sch=segment_f set_property -dict {
PACKAGE_PIN L18 IOSTANDARD LVC MOS33 } [get_ports { segment_g }]; IO_L4P_T0_D04_14
Sch=segment_g

set_property -dict { PACKAGE_PIN H15 IOSTANDARD LVC MOS33 } [get_ports { segment_dp }];
IO_L19N_T3_A21_VREF_15 Sch=segment_dp

set_property -dict { PACKAGE_PIN J17 IOSTANDARD LVC MOS33 } [get_ports { AN0 }];
IO_L23P_T3_FOE_B_15 Sch=an0 set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVC MOS33 }
[get_ports { AN1 }]; IO_L23N_T3_FWE_B_15 Sch=an1 set_property -dict { PACKAGE_PIN T9 IOSTANDARD
LVC MOS33 } [get_ports { AN2 }]; IO_L24P_T3_A01_D17_14 Sch=an2 set_property -dict { PACKAGE_PIN J14
IOSTANDARD LVC MOS33 } [get_ports { AN3 }]; IO_L19P_T3_A22_15 Sch=an3 set_property -dict {
PACKAGE_PIN P14 IOSTANDARD LVC MOS33 } [get_ports { AN4 }]; IO_L8N_T1_D12_14 Sch=an4
set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVC MOS33 } [get_ports { AN5 }];
IO_L14P_T2_SRCC_14 Sch=an5 set_property -dict { PACKAGE_PIN K2 IOSTANDARD LVC MOS33 }
[get_ports { AN6 }]; IO_L23P_T3_35 Sch=an6 set_property -dict { PACKAGE_PIN U13 IOSTANDARD
LVC MOS33 } [get_ports { AN7 }]; IO_L23N_T3_A02_D18_14 Sch=an7

LEDs

set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVC MOS33 } [get_ports { LED0 }];
IO_L18P_T2_A24_15 Sch=led0 set_property -dict { PACKAGE_PIN K15 IOSTANDARD LVC MOS33 }
[get_ports { LED1 }]; IO_L24P_T3_RS1_15 Sch=led1 set_property -dict { PACKAGE_PIN J13 IOSTANDARD
LVC MOS33 } [get_ports { LED2 }]; IO_L17N_T2_A25_15 Sch=led2 set_property -dict { PACKAGE_PIN N14
IOSTANDARD LVC MOS33 } [get_ports { LED3 }]; IO_L8P_T1_D11_14 Sch=led3 set_property -dict {
PACKAGE_PIN R18 IOSTANDARD LVC MOS33 } [get_ports { LED4 }]; IO_L7P_T1_D09_14 Sch=led4
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVC MOS33 } [get_ports { LED5 }];
IO_L18N_T2_A11_D27_14 Sch=led5 set_property -dict { PACKAGE_PIN U17 IOSTANDARD LVC MOS33 }
[get_ports { LED6 }]; #IO_L17P_T2_A14_D30_14 Sch=led6 set_property -dict { PACKAGE_PIN U16
IOSTANDARD LVC MOS33 } [get_ports { LED7 }]; #IO_L18P_T2_A12_D28_14 Sch=led7 set_property -dict {
PACKAGE_PIN V16 IOSTANDARD LVC MOS33 } [get_ports { LED8 }]; #IO_L16N_T2_A15_D31_14 Sch=led8
set_property -dict { PACKAGE_PIN T15 IOSTANDARD LVC MOS33 } [get_ports { LED9 }];
#IO_L14N_T2_SRCC_14 Sch=led9 set_property -dict { PACKAGE_PIN U14 IOSTANDARD LVC MOS33 }
[get_ports { LED10 }]; #IO_L22P_T3_A05_D21_14 Sch=led10 set_property -dict { PACKAGE_PIN T16
IOSTANDARD LVC MOS33 } [get_ports { LED11 }]; #IO_L15N_T2_DQS_DOUT_CSO_B_14 Sch=led11
set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVC MOS33 } [get_ports { LED12 }];
#IO_L16P_T2_CSI_B_14 Sch=led12 set_property -dict { PACKAGE_PIN V14 IOSTANDARD LVC MOS33 }
[get_ports { LED13 }]; #IO_L22N_T3_A04_D20_14 Sch=led13 set_property -dict { PACKAGE_PIN V12
IOSTANDARD LVC MOS33 } [get_ports { LED14 }]; #IO_L20N_T3_A07_D23_14 Sch=led14 set_property -dict
{ PACKAGE_PIN V11 IOSTANDARD LVC MOS33 } [get_ports { LED15 }]; #IO_L21N_T3_DQS_A06_D22_14
Sch=led15